

Technische Hochschule Deggendorf
Fakultät Angewandte Informatik
Studiengang 4. Semester Cyber Security (CY-B)

DroneBusta: Supply-Chain-Angriff auf SkyDefence AG

Vorgelegt von:

Christof Renner
Matrikelnummer: 22301943
Manuel Friedl
Matrikelnummer: 1236626
Jonas Gaßner
Matrikelnummer: 22307744

Prüfungsleitung:

Prof. Dr. Michael Heigl

Am: 15. Juni 2025

Inhaltsverzeichnis

1	Einleitung	2
1.1	Motivation	2
2	Szenario	3
2.1	Rahmenhandlung	3
2.2	Angriffsziele	3
2.3	Angriffsschritte	3
3	Vorbereitung	5
3.1	Zentrale GitLab-Repository	5
3.2	Aufsetzen des Docker-Containers	5
4	Supply-Chain-Angriffe	6
4.1	Grundprinzip	6
4.2	Bedeutung für Drohnen	6
5	Fiktives Angriffsszenario	7
5.1	Die Firmen im Überblick	7
5.2	Angriffsplan	7
6	Praktische Umsetzung	8
6.1	Aufbau des Custom-Modules	8
6.2	Simulation starten	8
7	Erkennung und Gegenmaßnahmen	9
7.1	Erkennung	9
7.2	Gegenmaßnahmen	9
8	Fazit	10

Kapitel 1

Einleitung

1.1 Motivation

In einer zunehmend digitalisierten Welt greifen zivile und militärische Anwendungen immer stärker auf unbemannte Luftfahrtsysteme zurück. Mit dieser Vernetzung wachsen jedoch auch die Angriffsflächen – insbesondere durch manipulierte Software in der Lieferkette, sogenannte *Supply-Chain-Angriffe*. Ziel dieses Projekts ist es, in einer sicheren Übungsumgebung genau diese Angriffsmethode nachzustellen. Wir nutzen dafür die **Damn Vulnerable Drone** Plattform, eingebettet in einen Docker-Container, der alle Komponenten in einer zentralen GitLab-Repository hostet.

Kapitel 2

Szenario

2.1 Rahmenhandlung

Hinweis: Dieses Szenario dient ausschließlich zu Lehrzwecken in einer sicheren Testumgebung. Jegliche Anwendung dieser Methoden auf fremde Systeme ohne ausdrückliche Erlaubnis ist illegal und strafbar!

Wichtig: Alle im Folgenden beschriebenen Schritte werden innerhalb eines Docker-Containers ausgeführt, der die *Damn Vulnerable Drone*-Plattform und ein speziell vorbereitetes Navigationsmodul enthält.

Sie gehören zur Angreifergruppe *DroneBusta*, einem fiktiven Team von Sicherheitsexperten, das im Auftrag einer unbekannten Organisation eine Serie von Drohnen der **SkyDefence AG** kompromittieren soll. SkyDefence AG ist ein international tätiger Hersteller von unbemannten Luftfahrtsystemen (UAS) für Logistik, Überwachung und Rettungseinsätze. Um Kosten zu sparen, hat SkyDefence AG die Entwicklung eines zentralen Navigationsmoduls an den Zulieferer **SkyDeliverer AG** ausgelagert. Dieses Modul, in Python implementiert und als Paket `navkit` verfügbar, wird in jeder Firmware-Version der Drohnen automatisch eingebunden.

2.2 Angriffsziele

- **Primäres Ziel:** Manipulation des Pakets `navkit`, sodass die Drohne bei Aufruf der Funktion `return_to_home()` eine kontrollierte Fehlfunktion auslöst und abstürzt.
- **Sekundäres Ziel:** Unbemerkte Verbreitung der manipulierten Bibliothek über die CI/CD-Pipeline von SkyDeliverer AG.

2.3 Angriffsschritte

1. Reconnaissance und Zugang:

- Per Social Engineering oder Phishing-Kampagne erlangen Sie Zugang zur GitLab-Instanz von SkyDeliverer AG.

- Sie klonen das offizielle Repository mit dem Originalcode des Moduls `navkit`.

2. Deployment im Testcontainer:

- Beim nächsten Start des Docker-Containers wird Ihre manipulierte Version von `navkit` verwendet.
- Sie starten die Simulation mittels

```
1 docker-compose up -d
2 docker exec -it dvd-simulation bash
```

3. Triggern des Angriffs im Simulator:

- In QGroundControl befehlen Sie „Return to Home“.
- Die Schadroutine aktiviert den MAVLink-Befehl `DO_FLIGHT_TERMINATION` und verursacht den simulierten Absturz der Drohne.

Kapitel 3

Vorbereitung

3.1 Zentrale GitLab-Repository

Alle Projektmaterialien befinden sich in einer zentralen GitLab-Instanz. Dort liegen:

- Das Original-Repository der *Damn Vulnerable Drone* — inkl. aller Docker-Konfigurationen
- Ein Verzeichnis `custom_module/`, das die Ausgangsversion des zu modifizierenden Moduls enthält
- Ausführliche Anleitungen, Beispielskripte und Konfigurationsdateien

3.2 Aufsetzen des Docker-Containers

1. Repository klonen:

```
1 git clone https://gitlab.th-deg.de/cyb/project -  
   dronebusta.git  
2 cd project-dronebusta
```

2. Container starten:

```
1 docker-compose up -d
```

3. Zugriff auf die Shell des Containers:

```
1 docker exec -it dvd-simulation bash
```

4. Verzeichnisstruktur im Container prüfen:

```
1 ls /opt/dvd  
2 ls /opt/custom_module
```

Kapitel 4

Supply-Chain-Angriffe

4.1 Grundprinzip

Ein Supply-Chain-Angriff infiltriert Softwarepakete oder Bibliotheken bei deren Erstellung, bevor sie in das Zielsystem gelangen. Der Angreifer schaltet in einer Phase dazwischen, in der Entwickler oder CI/CD-Pipelines Pakete zusammenstellen und weiterverteilen.

4.2 Bedeutung für Drohnen

Da Drohnensoftware häufig auf Open-Source-Bibliotheken und externen Modulen aufbaut, ist jede Komponente potentieller Einfallstor. Ein einziger fehlerhafter Build kann tausende Drohnen weltweit kompromittieren.

Kapitel 5

Fiktives Angriffsszenario

5.1 Die Firmen im Überblick

- **SkyDefence AG:** Hersteller sicherheitskritischer Drohnen für industrielle und militärische Einsätze.
- **SkyDeliverer AG:** Externer Zulieferer, der ein wichtiges Navigationsmodul in Python entwickelt.
- **DroneBusta:** Hackergruppe, die im Namen einer konkurrierenden Nation die Drohnen sabotiert.

5.2 Angriffsplan

1. *Präparate:* Einspeisung eines Backdoors in das Python-Modul `navkit.py`.
2. *Distribution:* Veröffentlichung der manipulierten Version ins GitLab-Repo der SkyDeliverer AG (im Szenario per Social Engineering erlangt).
3. *Deployment:* Docker liefert automatisch die bösartige Version in die DVD-Simulation.
4. *Activation:* Trigger im Code, der beim Aufruf von `return_to_home()` eine Crash-Sequenz auslöst.

Kapitel 6

Praktische Umsetzung

6.1 Aufbau des Custom-Modules

Öffne im Container das Verzeichnis und bearbeite die Datei `navkit.py`:

```
1 def return_to_home():
2     # Standard-Logik
3     fly_home()
4     # Schad-Routine
5     crash_drone()
6
7 def crash_drone():
8     # MAVLink-Befehl zum Auslösen eines Kontrollverlusts
9     mav.send_command('DO_FLIGHTTERMINATION')
```

6.2 Simulation starten

1. Gazebo-Welt laden:

```
1 gazebo /opt/dvd/worlds/empty_world.world
```

2. DVD-Firmware starten:

```
1 cd /opt/dvd
2 ./run_dvd.sh
```

3. MAVLink-Monitor öffnen:

```
1 mavlink_tool --listen 14550
```

4. Im QGroundControl per `RETURN_TO_HOME` das Rückkehrmanöver auslösen und beobachten, wie die Drohne abstürzt.

Kapitel 7

Erkennung und Gegenmaßnahmen

7.1 Erkennung

- Protokollanalyse: Ungewöhnliche MAVLink-Befehle (`DO_FLIGHTTERMINATION`).
- Datei-Integrität: Änderung der Prüfsumme des Moduls `navkit.py`.

7.2 Gegenmaßnahmen

- Signierung von Modulen vor Deployment.
- Strenge Code-Reviews und automatisierte Tests in CI/CD.
- Monitoring auf anomale Telemetrie-Muster.

Kapitel 8

Fazit

Mit diesem Szenario haben die Teilnehmenden erlebt, wie ein einzelnes manipuliertes Modul eine ganze Drohne zum Absturz bringen kann. Die Übung stärkt das Bewusstsein für Supply-Chain-Risiken und zeigt praktikable Abwehrstrategien auf.